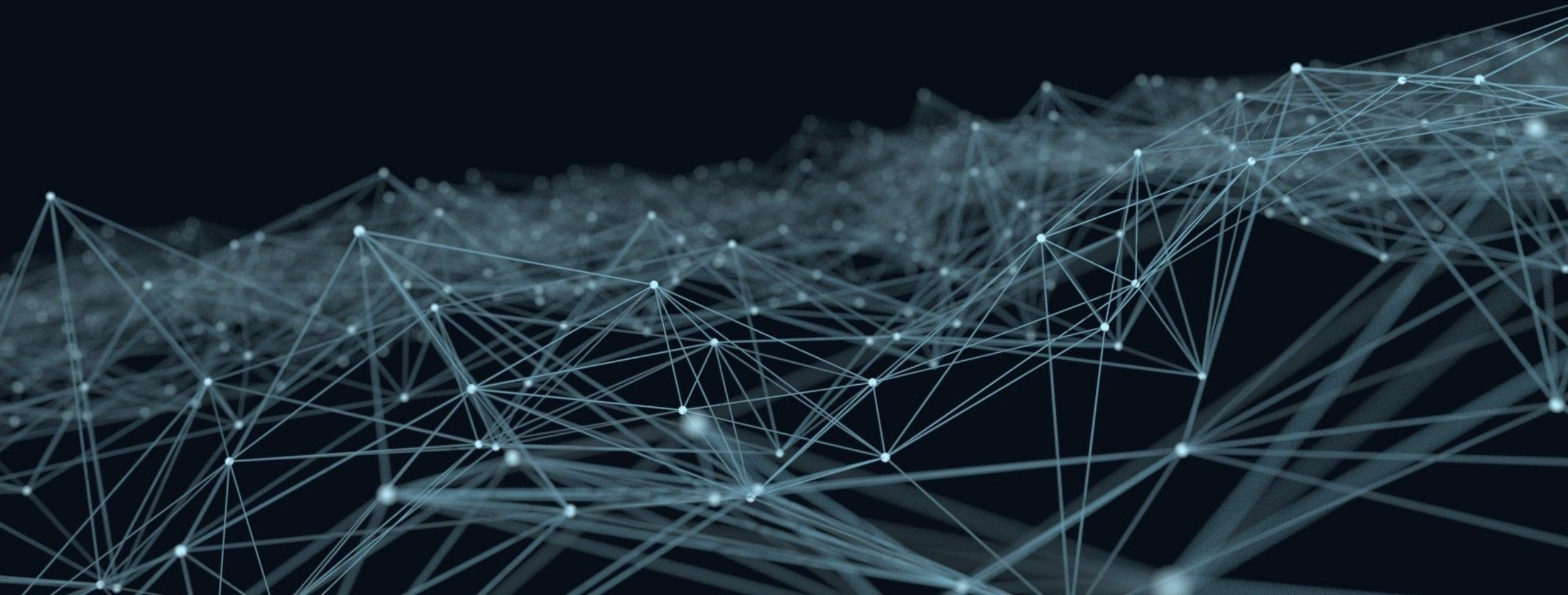
Java Full Stack Development

4. React Basics



Module Topics

- Introduction to React
- React Architecture Overview
- Functional Components and JSX
- Hooks
- State Management
- Project Structure

Event-Driven Architecture

Browser as an Application Platform

- Era 1 - Server-rendered pages
  - Every interaction is a full page reload
  - Server owns all state and rendering
- Era 2 - AJAX-enhanced pages
  - Server still renders,
  - JavaScript can fetch fragments asynchronously.

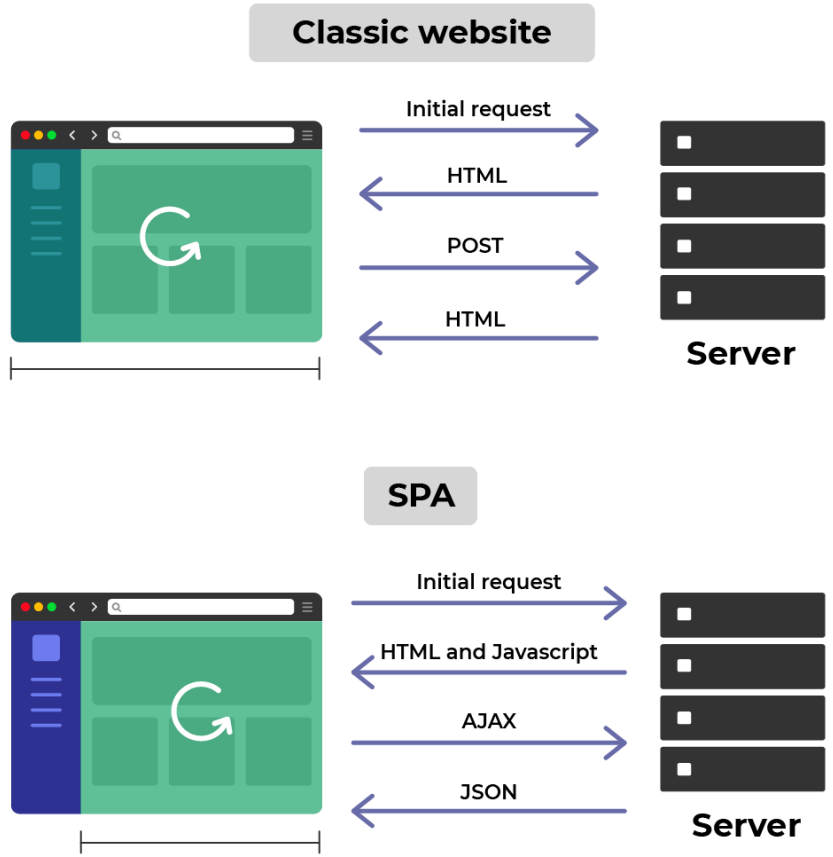


Image Credit: https://openclassrooms.com/en/courses/7315991-intermediate-react/7572037-turn-your-application-into-a-single-page-app-with-react-router

Browser as an Application Platform

- Era 3 - Single-Page Applications
  - Browser owns rendering
  - Server becomes a JSON API.
- Era 4 - SSR / hybrid
  - Server renders the first response for performance and SEO
  - The browser “hydrates” and takes over
    - Server sends down HTML
    - Browser attaches event handlers, component state and other functionality
- Each era was a response to the limitations of the previous eras

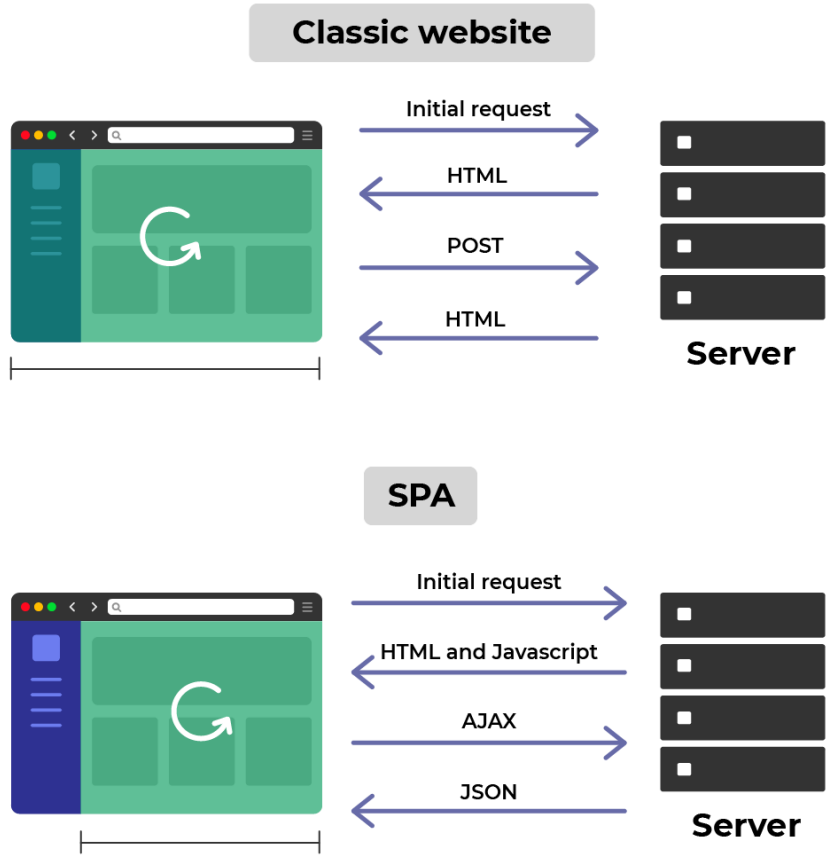


Image Credit: https://openclassrooms.com/en/courses/7315991-intermediate-react/7572037-turn-your-application-into-a-single-page-app-with-react-router

SPA

- Single page application
  - Web app that loads one HTML document
  - Uses JavaScript to handle
    - All subsequent rendering and navigation in the browser
    - Fetching data from APIs instead of requesting new pages from the server
- Benefits
  - Responsive UI
    - Interactions don't wait for a network round-trip to render
  - Decoupled lifecycles
    - Frontend and backend deploy independently

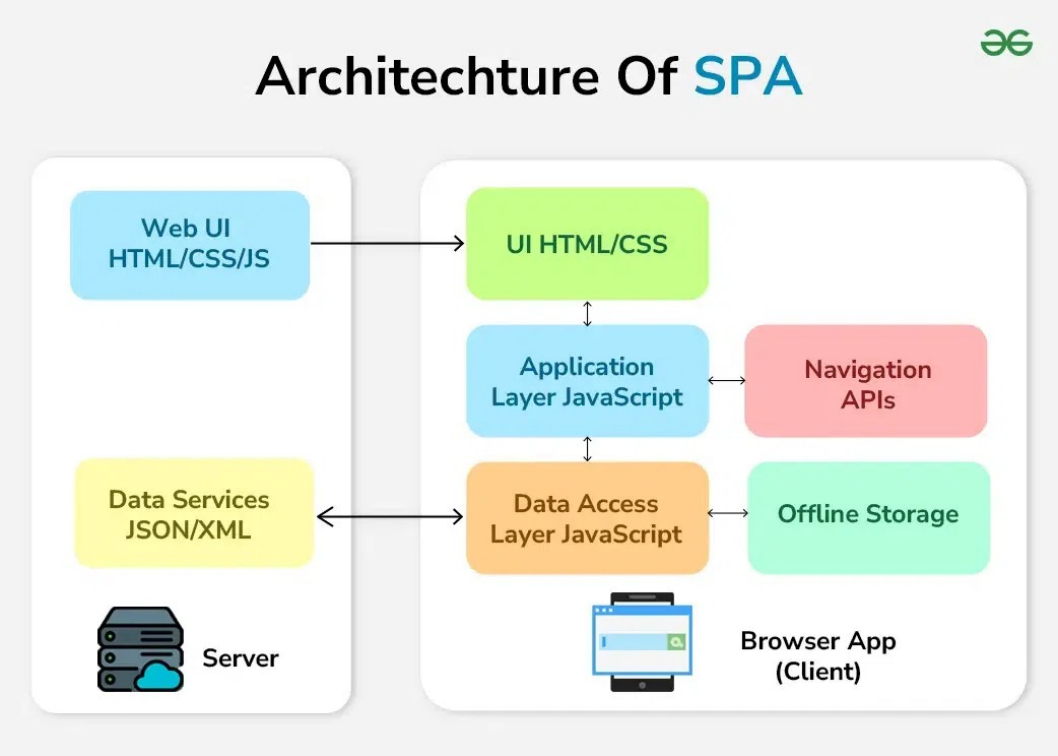


Image Credit: https://www.geeksforgeeks.org/reactjs/what-are-single-page-apps/

SPA

- Benefits
  - Clean separation of concerns
    - Backend is a pure API
    - Frontend owns presentation
  - Reusable APIs
    - Same backend serves web, mobile, partners, internal tools
  - Rich client experiences
    - Drag-and-drop, real-time updates, offline support

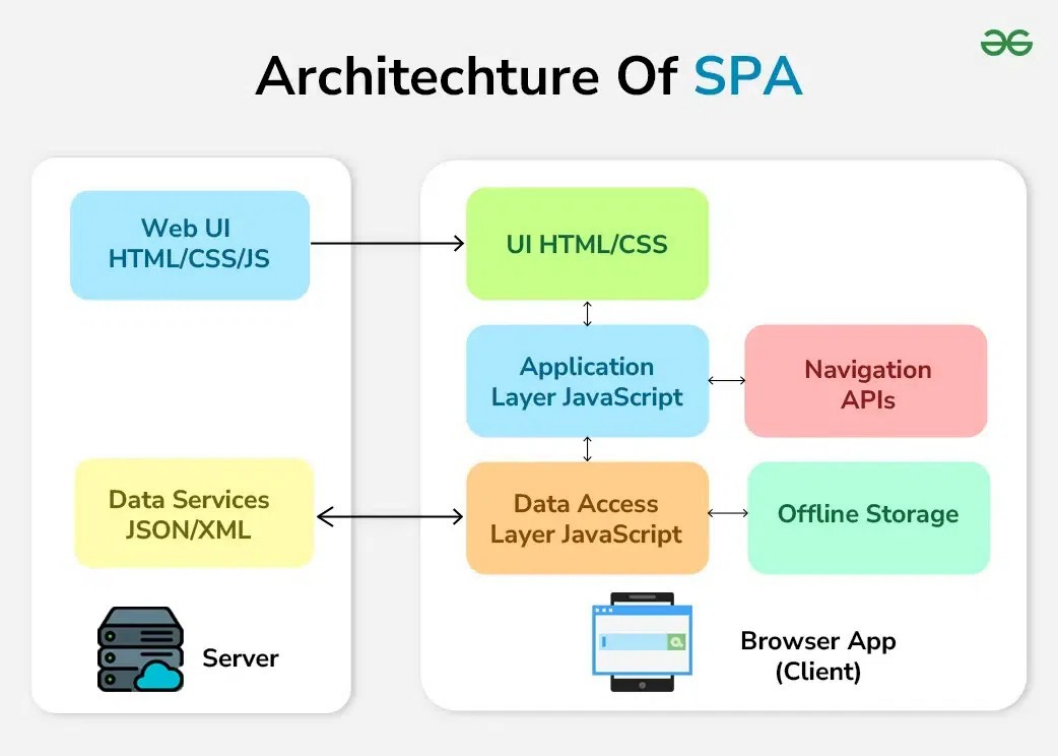


Image Credit: https://www.geeksforgeeks.org/reactjs/what-are-single-page-apps/

SPA

- Downsides
  - Authentication complexity
    - No server session by default
    - Tokens in the browser
  - SEO trade-offs
    - Initial HTML is empty so search engines must execute JavaScript
  - Larger initial download
    - The entire app ships before anything renders
  - Client-side complexity
    - State management, routing, error handling all move to the browser

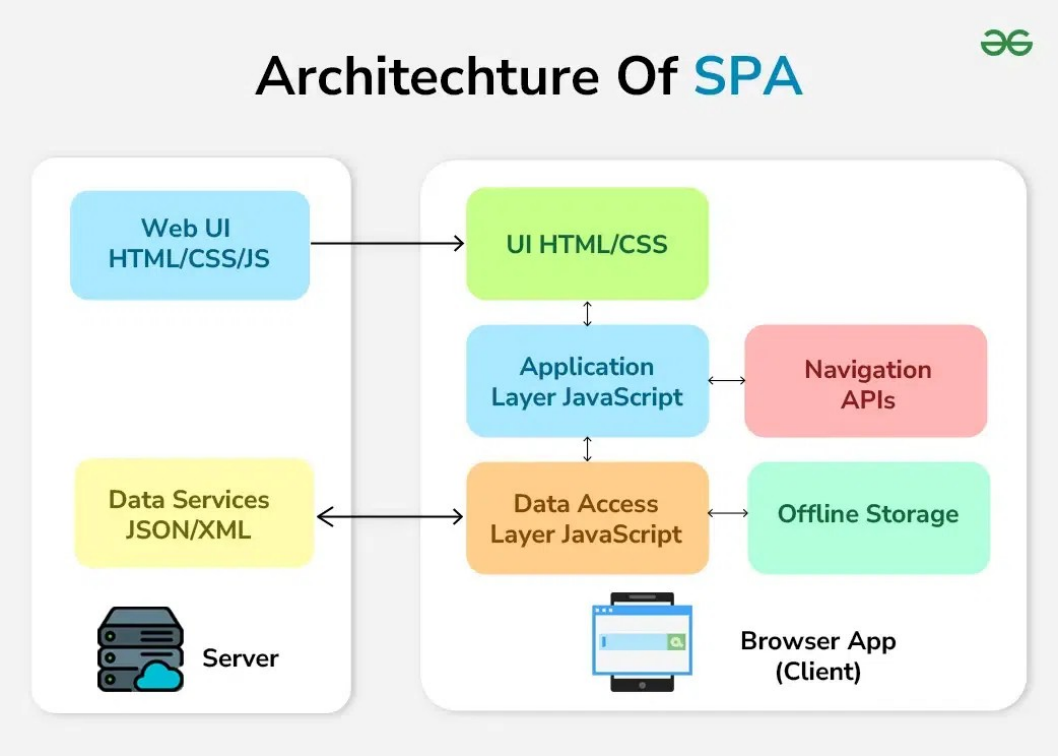


Image Credit: https://www.geeksforgeeks.org/reactjs/what-are-single-page-apps/

Where React Fits

- Specific role
  - A declarative, component-based model for describing UI
  - UI is a function of state
    - Describe what it should look like, React figures out how
  - Focused on the view layer
    - Not a full framework like Angular
  - A mental model, not a full toolkit
    - You assemble routing, data fetching, etc. yourself
  - Backed by a large ecosystem and a stable API across major versions

```
import React from 'react';
import {Text, View} from 'react-native';
import {Header} from './Header';
import {heading} from './Typography';

const WelcomeScreen = () => (
  <View>
    <Header title="Welcome to React Native"/>
    <Text style={heading}>Step One</Text>
    <Text>
      Edit App.js to change this screen and turn it
      into your app.
    </Text>
    <Text style={heading}>See Your Changes</Text>
    <Text>
      Press Cmd + R inside the simulator to reload
      your app's code.
    </Text>
    <Text style={heading}>Debug</Text>
    <Text>
```


Image Credit: https://simpat.tech/what-is-react/

Deployment Model

- How a React SPA is actually deployed
  - Build step produces static files
    - index.html, JavaScript bundles, CSS, assets
  - Hosted on any static file server
    - CDN, object storage, nginx
  - No runtime on the server for the SPA itself
    - No Node.js, no app server
  - The browser downloads the bundle once per session (with caching)
  - Backend APIs are separate deployments at separate URLs

```
import React from 'react';
import {Text, View} from 'react-native';
import {Header} from './Header';
import {heading} from './Typography';

const WelcomeScreen = () => (
  <View>
    <Header title="Welcome to React Native"/>
    <Text style={heading}>Step One</Text>
    <Text>
      Edit App.js to change this screen and turn it
      into your app.
    </Text>
    <Text style={heading}>See Your Changes</Text>
    <Text>
      Press Cmd + R inside the simulator to reload
      your app's code.
    </Text>
    <Text style={heading}>Debug</Text>
    <Text>
```


Image Credit: https://simpat.tech/what-is-react/

Security

- The SPA is a public client
  - The bundle is downloaded by every user
    - No client credentials or secrets unique to that client
  - Code runs in a user-controlled environment
    - Includnes DevTools, browser extensions, hostile networks
  - Any data the SPA holds is observable by the user and by malicious scripts
  - Authentication must work without trusting the client

```
import React from 'react';
import {Text, View} from 'react-native';
import {Header} from './Header';
import {heading} from './Typography';

const WelcomeScreen = () => (
  <View>
    <Header title="Welcome to React Native"/>
    <Text style={heading}>Step One</Text>
    <Text>
      Edit App.js to change this screen and turn it
      into your app.
    </Text>
    <Text style={heading}>See Your Changes</Text>
    <Text>
      Press Cmd + R inside the simulator to reload
      your app's code.
    </Text>
    <Text style={heading}>Debug</Text>
    <Text>
```


Image Credit: https://simpat.tech/what-is-react/

Security

- The SPA is a public client
  - Why we use PKCE instead of relying on a client secret
    - There is no client secret to rely on.
  - Tokens have to be stored somewhere the JavaScript can reach
    - Means they're vulnerable to anything that compromises the JavaScript
  - XSS is catastrophic for SPAs
    - More than for server-rendered apps
    - XSS in a server-rendered app might steal a session cookie
    - XSS in an SPA can drain the entire token store and impersonate the user freely.

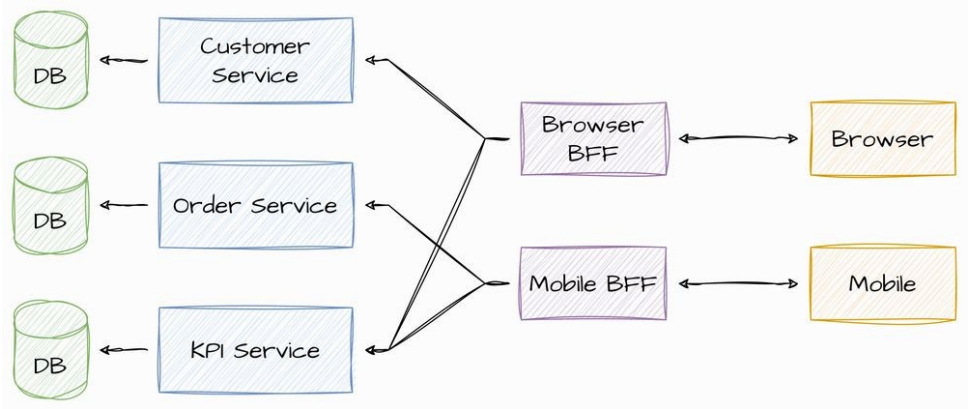
```
import React from 'react';
import {Text, View} from 'react-native';
import {Header} from './Header';
import {heading} from './Typography';

const WelcomeScreen = () => (
  <View>
    <Header title="Welcome to React Native"/>
    <Text style={heading}>Step One</Text>
    <Text>
      Edit App.js to change this screen and turn it
      into your app.
    </Text>
    <Text style={heading}>See Your Changes</Text>
    <Text>
      Press Cmd + R inside the simulator to reload
      your app's code.
    </Text>
    <Text style={heading}>Debug</Text>
    <Text>
```


Image Credit: https://simpat.tech/what-is-react/

Backend-for-Frontend

- The SPA is a public client
  - Why the BFF pattern exists
    - To move the secrets and tokens back to a confidential client (a Spring Boot backend)
    - And give the browser only an HttpOnly cookie
  - The BFF is "for" a specific frontend
    - A mobile app, it gets its own BFF tailored to its needs for example
  - Different clients have different needs, so give each one a backend shaped to its needs




```
Browser (React SPA)  →  BFF (Spring Boot)  →  Resource APIs
                                           →  Other services
                                           →  Authorization Server
```


Image Credit: https://aien.me/simplifying-complex-systems-with-backend-for-frontends-bff/

Backend-for-Frontend

- The browser becomes "dumb"
  - From an auth perspective
  - It has a session cookie
  - The cookie identifies it to the BFF
  - The BFF, on the server side, holds the actual OAuth tokens and attaches them to outbound calls to the real APIs
  - Decouples the UI from the client logic

Concern	Pure-SPA approach	BFF approach
Who is the OAuth client?	The React SPA	The Spring Boot BFF
Where do tokens live?	In browser JavaScript	On the BFF server
What does the browser hold?	Access + refresh tokens	An HttpOnly session cookie
What can XSS steal?	Everything	Nothing useful — cookie is HttpOnly
Who calls the resource APIs?	The browser, with a bearer token	The BFF, with a bearer token

Backend-for-Frontend

- A BFF does two things:
  - Receives requests from the browser
    - Handles the session cookie, validates CSRF, routes to the right downstream API
  - Makes outbound calls to downstream APIs
    - Attaches the bearer token, awaits the response, returns it to the browser
- WebClient
  - pring's reactive, non-blocking HTTP client
  - It belongs to the outbound half
  - It's what the BFF uses to call the downstream resource servers.

Concern	Component
Receive HTTP requests from browser	Spring MVC `@RestController` (or WebFlux `@RestController`)
Session and cookie management	Spring Security session management
OAuth client (PKCE flow, token storage, refresh)	`spring-boot-starter-oauth2-client`
Outbound calls to resource APIs	`WebClient` (or `RestClient`, see below)
CSRF protection	Spring Security `CookieCsrfTokenRepository`

Backend-for-Frontend

- Simple conceptual model
  - React UI interacts with the user
  - Talks to the WebClient backend
    - Asks for a specific URL
  - Webclient holds credentials
    - Attaches credentials to the request
  - Webclient makes the request to the server
    - With the included tokes or credentials

Concern	Component
Receive HTTP requests from browser	Spring MVC `@RestController` (or WebFlux `@RestController`)
Session and cookie management	Spring Security session management
OAuth client (PKCE flow, token storage, refresh)	`spring-boot-starter-oauth2-client`
Outbound calls to resource APIs	`WebClient` (or `RestClient`, see below)
CSRF protection	Spring Security `CookieCsrfTokenRepository`

React Architecture Overview

The Component

- The fundamental unit
  - A component is a function that takes properties (props) and returns a description of UI
  - Components compose
    - They call other components the way functions call functions
  - Components are isolated
    - They own their state and don't reach into other components
  - The whole application is a tree of components rooted at a single top-level component
  - Everything else in React is in service of making components work well

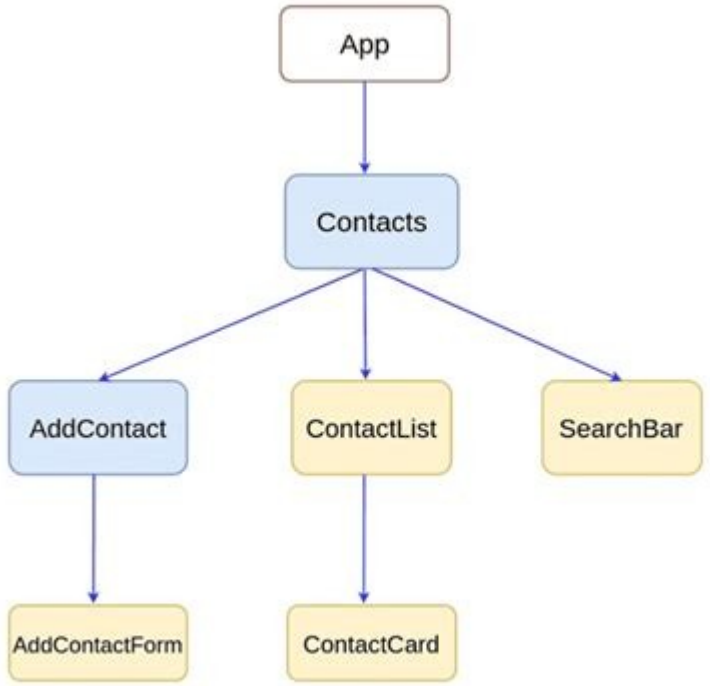


Image Credit: https://www.asktheegghead.com/react-js-for-wordpress-users-a-basic-introduction/

The Component

- Nothing more than
  - A typed function taking props as its argument
  - Returns JSX
    - Description of UI, not actual DOM
    - Converted into DOM elements
- Used as a custom HTML-like tag elsewhere in the tree
  - Components start with uppercase letter
  - HTML tags start all lowercase
  - Improves readability

```tsx
type GreetingProps = {
  name: string;
  enthusiastic?: boolean;
};

function Greeting({ name, enthusiastic = false }: GreetingProps) {
  return <p>Hello, {name}{enthusiastic ? '!' : '.'}</p>;
}

// Used like this:
<Greeting name="Ada" enthusiastic />
```

The Component

- GreetingProps
  - Every component's props are typed
    - Supported by TypeScript
  - Example uses destructured props with a default value
- The return value is JSX
  - ... not a string and not a DOM
    - Syntax that compiles to a JavaScript function call
- Usage
  - Components are used as if they were custom HTML tags
  - The compiler turns this into a function call

```tsx
type GreetingProps = {
  name: string;
  enthusiastic?: boolean;
};

function Greeting({ name, enthusiastic = false }: GreetingProps) {
  return <p>Hello, {name}{enthusiastic ? '!' : '.'}</p>;
}

// Used like this:
<Greeting name="Ada" enthusiastic />
```

Declarative vs. Imperative

- Imperative
  - Find the element, change its text, add a class
- Declarative
  - Describe the destination
    - "This is what the UI should look like right now"
  - React figures out the steps to get from the current screen to the described destination
  - You write what the outcome should be
  - React handles how to produce the outcome

```
$('#badge').text(count);
if (count > 0) {
  $('#badge').addClass('has-items');
} else {
  $('#badge').removeClass('has-items');
}
```

```
<span className={count > 0 ? 'badge has-items' : 'badge'}>{count}</span>
```

UI State

- UI as a Function of State
  - UI = f(state)
  - Same state in results in the same UI out
    - Functional programming term: pure function
  - To change the UI, change the state
    - React re-runs the function when state changes
  - Re-running produces a new description
    - React figures out what changed

state is used for internal communication inside a Component

Image Credit: https://iq.js.org/questions/react/what-is-state-in-react

UI State

- Components should be deterministic
  - Given the same props and state, they should produce the same output
    - They shouldn't depend on hidden globals
    - They shouldn't have side effects during rendering
    - They shouldn't mutate their inputs.
  - When state changes
    - React calls the component function again
    - You don't tell it which DOM nodes to update
    - You just produce a fresh description, and React handles the rest

state is used for internal communication inside a Component

Image Credit: https://iq.js.org/questions/react/what-is-state-in-react

The Virtual DOM

- Virtual DOM
  - A plain JavaScript object tree describing what the UI should look like
  - Created when your component function runs and returns JSX
  - Cheap to create, cheap to throw away
  - Lives in memory only
    - Not connected to the real DOM
  - Each render produces a fresh virtual DOM tree

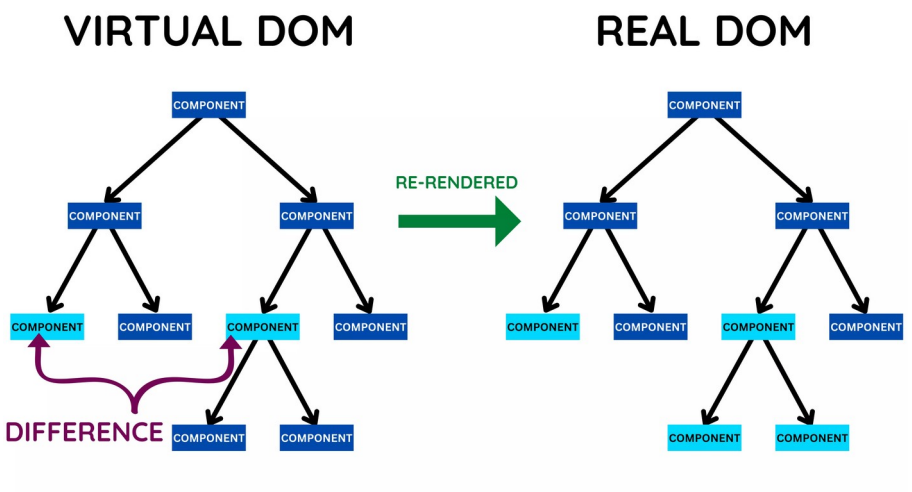


Image Credit:https://skillzam.com/notes/reactjsNotes.html

The Virtual DOM

- Virtual DOM
  - JSX compiles into calls that produce objects shaped like the one shown
    - They're plain JavaScript, they live in memory, and creating them is cheap
  - The real DOM is expensive
    - Every DOM mutation can trigger layout recalculation, style recalculation, and repaint
    - Doing many DOM operations is slow
    - Manipulating plain JavaScript objects is much faster than manipulating DOM nodes

```
{
  type: 'div',
  props: { className: 'card' },
  children: [
    { type: 'h2', props: {}, children: ['Title'] },
    { type: 'p', props: {}, children: ['Body'] }
  ]
}
```

The Virtual DOM

- React strategy
  - When your state changes
  - Build a new cheap virtual DOM tree
  - Compare it against the previous virtual DOM tree
    - A cheap operation
  - Figure out the minimum set of real DOM changes needed
    - Usually a small set
  - Apply only the needed changes
  - The expensive part involving the real DOM tree, is minimized

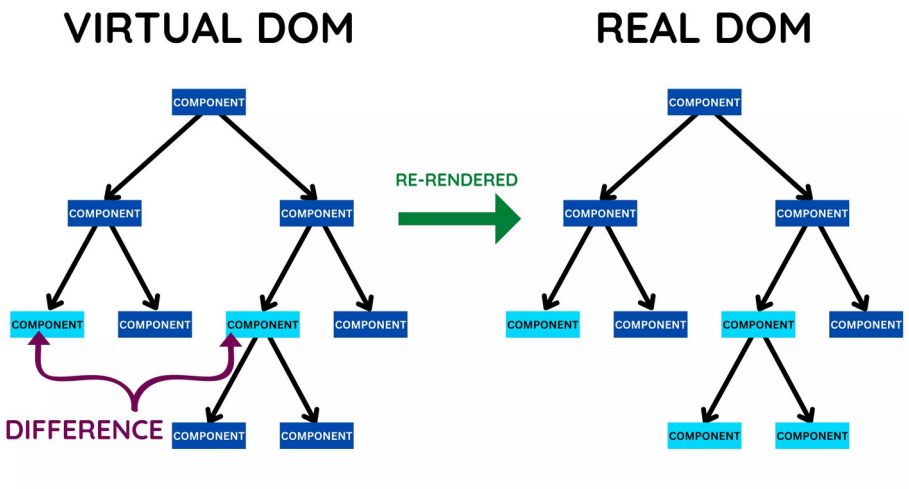


Image Credit:https://skillzam.com/notes/reactjsNotes.html

The Virtual DOM

- Virtual DOM
  - Not a copy of the entire real DOM
  - Not the source of React's speed by itself
    - It's the diffing strategy that matters
  - Not something you interact with directly
    - You write JSX and React handles it
  - Not unique to React
    - Vue and others use similar approaches
  - Not always faster than direct DOM manipulation
    - But it's predictable

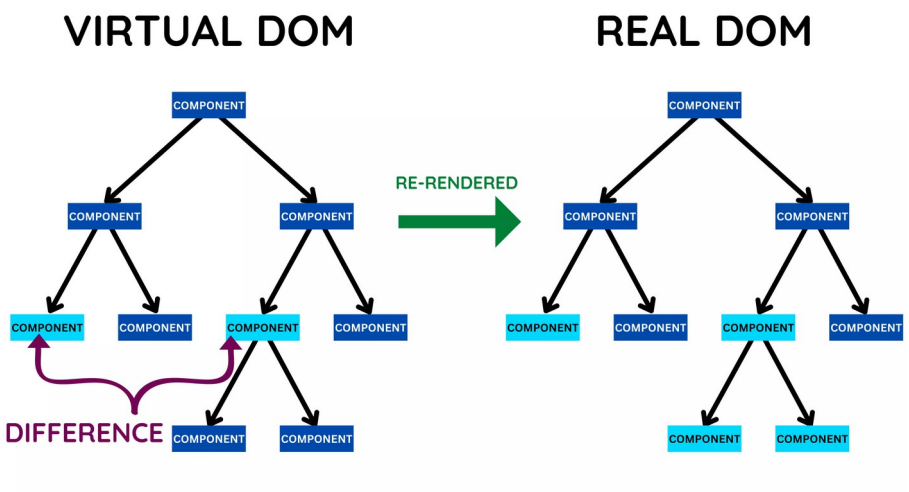


Image Credit:https://skillzam.com/notes/reactjsNotes.html

Reconciliation

- Diffing the trees (reconciliation)
  - React compares the new virtual DOM to the previous virtual DOM
    - Walks both trees in parallel, looking for differences
    - Changes are applied to the real DOM as a minimal set of operations
  - Same node type
    - Just update props in place
  - Different node type
    - Unmount old subtree
    - Mount new subtree

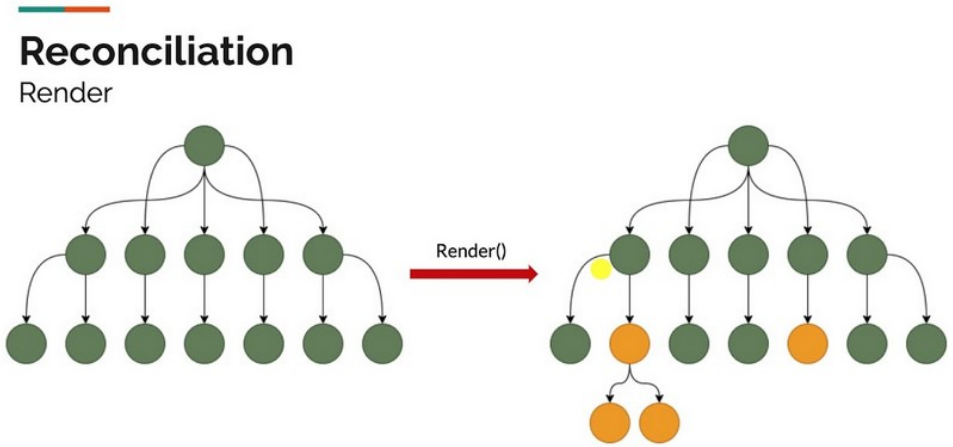


Image Credit: https://www.youtube.com/watch?v=Ugdv6GIEx14

Reconciliation

- The key rules
  - Same component or element type at the same position
    - React assumes it's the "same" node and updates its props in place
    - The DOM node is preserved, internal state is preserved, only the changed attributes are updated
  - Different type at the same position
    - React assumes it's a completely different thing
    - It unmounts the old subtree (running cleanup, destroying DOM nodes) and mounts a fresh one
  - Children of the same parent
    - Matched by position by default

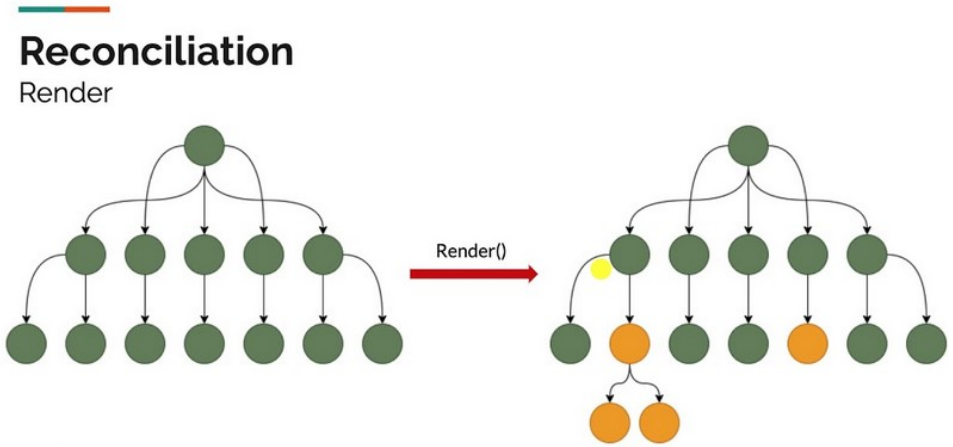


Image Credit: https://www.youtube.com/watch?v=Ugdv6GIEx14

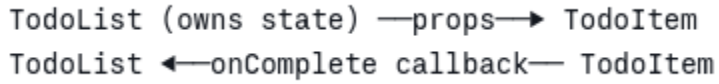
Reconciliation

- Keys and list reconciliation
  - Without keys, React matches list children by position
    - Reordering or inserting at the front looks like "every item changed"
  - Keys tell React the identity of each item across renders
    - Use a stable, unique ID
    - Never the array index for dynamic lists
  - Wrong keys
    - Subtle bugs
    - lost form state

```
{users.map(user => (
  <UserCard key={user.id} user={user} />
))}
```

Flow

- Unidirectional data flow
  - Data flows downward through the tree as props
  - A child cannot reach into a parent or sibling to read or mutate state
  - To communicate upward, a child invokes a callback passed as a prop
  - The parent owns the state; the child renders it and reports interactions
  - Data has a single source of truth at each level



Render Phase

- Rendering
  - A render is calling the component function with current props and state
  - Returns a new virtual DOM tree
  - Renders are fast and frequent
    - Not the same as updating the screen
  - Rendering itself does not touch the DOM
  - Triggered when
    - The component's own state changes
    - The component's props change because the parent re-rendered
    - A context value the component subscribes to changes
    - The parent re-rendered

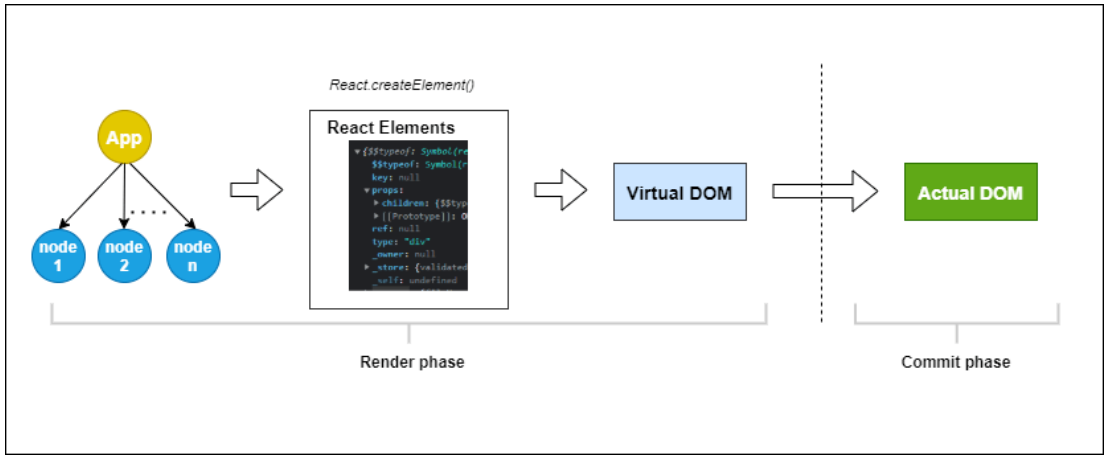


Image Credit: https://www.telerik.com/blogs/understand-how-rendering-works-react

Render vs. Commit

- Two phases
  - Render phase
    - Call components
    - Build new virtual DOM
    - Diff against old
    - Pure, no side effects
    - May be paused or thrown away by React.
  - Commit phase
    - Apply the diffed changes to the real DOM
    - Synchronous, fast, runs effects.
  - A render does not guarantee a commit
    - React may decide nothing needs updating

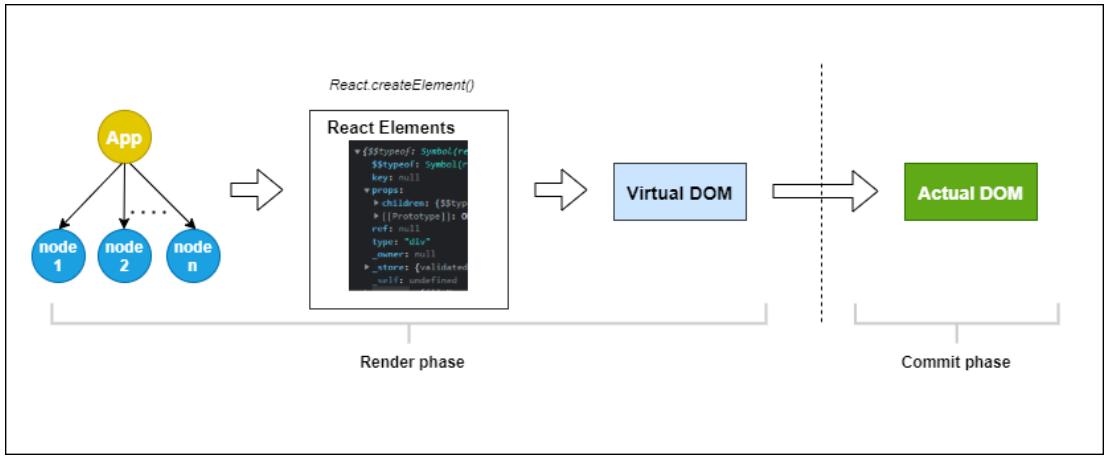


Image Credit: https://www.telerik.com/blogs/understand-how-rendering-works-react

Render vs. Commit

- Full loop
  - User interaction triggers state change
  - State change causes React to schedule a render
  - Render phase causes component functions runs, returns new virtual DOM
  - Reconciliation does a diff against previous virtual DOM
  - In the commit phase minimal DOM changes applied, effects run
  - Browser then paints the updated screen

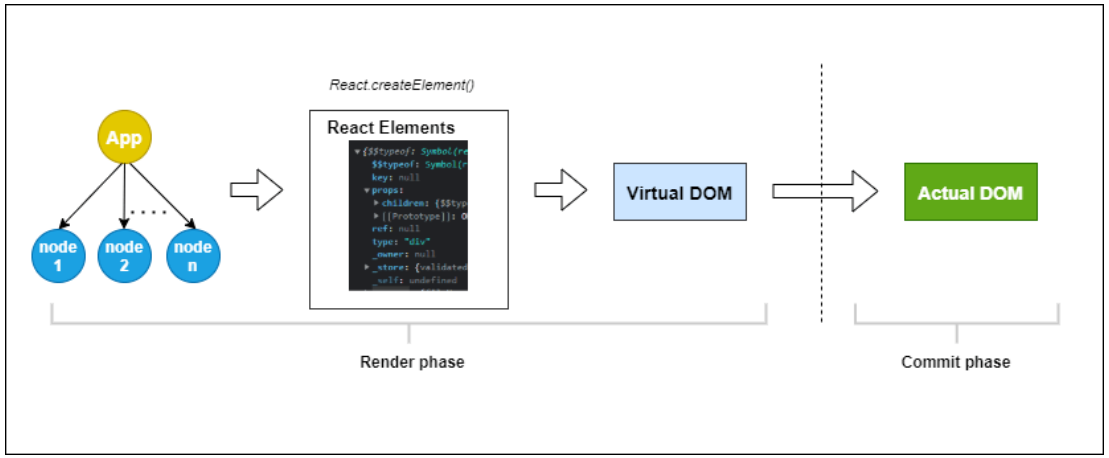


Image Credit: https://www.telerik.com/blogs/understand-how-rendering-works-react

Functional Components and JSX

Components

- Typed props as a type alias
- Destructured arguments with default values
- Hook call at the top of the function
  - Covered in the next section
- JSX return value describing the UI
- Named export so other components can import it

```tsx
import { useState } from 'react';

type CounterProps = {
  initial?: number;
  label: string;
};

export function Counter({ initial = 0, label }: CounterProps) {
  const [count, setCount] = useState(initial);

  return (
    <div className="counter">
      <span>{label}: {count}</span>
      <button onClick={() => setCount(count + 1)}>+</button>
    </div>
  );
}
```

JSX

- JSX looks like HTML, but isn't
  - Syntactic sugar over JavaScript function calls
    -
    - compiles to
    - React.createElement('div', { className: 'x' })
  - The compiler handles the transformation at build time
  - The browser never sees JSX, only plain JavaScript
  - Differences from HTML are deliberate and consistent

```jsx
<div className="card">
  <h2>Hello</h2>
  <p>{name}</p>
</div>
```

```jsx
React.createElement('div', { className: 'card' },
  React.createElement('h2', null, 'Hello'),
  React.createElement('p', null, name)
);
```

JSX

- Expressions in JSX
  - Curly braces { } mean "evaluate this JavaScript expression"
  - Any expression is valid
    - Variables, function calls, ternaries, arithmetic
  - Statements are not allowed
    - No if, no for, no let
  - null, undefined, false, and true render as nothing
  - Strings and numbers render as text

```
<p>Hello, {user.name}</p>
<p>You have {messages.length} unread {messages.length === 1 ? 'message' : 'messages'}</p>
<p>{user.avatar && <img src={user.avatar} alt="" />}</p>
```

JSX

- Three things happen inside curly braces
  - The expression is evaluated.
  - The result is converted to something React can render
  - That something becomes a child of the surrounding JSX node

```
<p>Hello, {user.name}</p>
<p>You have {messages.length} unread {messages.length === 1 ? 'message' : 'messages'}</p>
<p>{user.avatar && <img src={user.avatar} alt="" />}</p>
```

JSX

- Something React can render means
  - Strings → rendered as text.
  - Numbers → rendered as text.
  - React elements (other JSX) → rendered as children
  - Arrays of the above → rendered in order (this is how list rendering works)
  - null, undefined, false, true → render nothing
    - This is what makes the {condition && <Thing />} pattern work.
  - Objects → throw an error
    - You can't render {user} directly; you have to extract specific properties

```
<p>Hello, {user.name}</p>
<p>You have {messages.length} unread {messages.length === 1 ? 'message' : 'messages'}</p>
<p>{user.avatar && <img src={user.avatar} alt="" />}</p>
```

Conditional Rendering

- Three patterns
  - Ternary for two-branch decisions inline
  - Short-circuit (&&) when the false case renders nothing
  - Early return when the entire component output depends on the condition

```
// Ternary — when there are two branches
{isLoggedIn ? <Dashboard /> : <LoginPrompt />}

// Short-circuit — when the alternative is "render nothing"
{hasErrors && <ErrorBanner errors={errors} />}

// Early return — when the whole component is conditional
function UserProfile({ user }: { user?: User }) {
  if (!user) return <Spinner />;
  return <div>...</div>;
}
```

Rendering Lists

- Rendering collections
  - Use Array.prototype.map to transform data into JSX
  - Each item must have a unique key prop
  - Keys must be stable and come from the data, not the array index
  - Returning an array from a component is allowed but uncommon
  - For empty lists, return a fallback or render nothing
  - For empty lists, the conventional pattern is is shown on the right

```tsx
type UserListProps = {
  users: User[];
};

function UserList({ users }: UserListProps) {
  return (
    <ul>
      {users.map(user => (
        <li key={user.id}>
          {user.name}
        </li>
      ))}
    </ul>
  );
}
```

```tsx
{users.length === 0
  ? <EmptyState />
  : <ul>{users.map(user => <li key={user.id}>{user.name}</li>)}</ul>}
```

Props in Depth

- Props
  - Are the inputs to a component
    - They are read-only inside the component
  - TypeScript types make props self-documenting and catch errors at compile time
  - Use literal union types ('primary' | 'secondary') to constrain valid values
  - Optional props use ? and typically have defaults

```
type ButtonProps = {
        label: string;
variant?: 'primary' | 'secondary';
disabled?: boolean;
onClick: () => void;
};

function Button({ label, variant = 'primary', disabled, onClick }: ButtonProps) {
        return (
    <button className={`btn btn-${variant}`} disabled={disabled} onClick={onClick}>
        {label}
    </button>
        );
        }

// Used like:
<Button label="Save" onClick={handleSave} />
<Button label="Delete" variant="secondary" onClick={handleDelete} disabled={isDeleting} />
```

Props in Depth

- Callback Props
  - Callback props are named with an on prefix
    - onClick, onChange, onSubmit.
  - Their type is a function type
    - () => void for callbacks with no arguments
    - (value: string) => void for those that pass data up.
  - The parent passes a function
    - The child invokes it when the relevant event happens

```
type ButtonProps = {
        label: string;
variant?: 'primary' | 'secondary';
disabled?: boolean;
onClick: () => void;
};

function Button({ label, variant = 'primary', disabled, onClick }: ButtonProps) {
        return (
    <button className={`btn btn-${variant}`} disabled={disabled} onClick={onClick}>
        {label}
    </button>
        );
        }

// Used like:
<Button label="Save" onClick={handleSave} />
<Button label="Delete" variant="secondary" onClick={handleDelete} disabled={isDeleting} />
```

Props in Depth

- Callback Prop
  - A function that a parent passes down to a child so the child can tell the parent when something happened
  - Rule
    - Props flow down, events bubble up
    - Data flows from parent to child as props
    - Children can't reach up and modify the parent's state directly
  - How does a child tell its parent "the user clicked me" or "the user typed something"?
    - The parent gives the child a function
    - The child calls it

```
// Parent owns the state and gives the child a way to update it
function Parent() {
  const [count, setCount] = useState(0);
  return <Button onClick={() => setCount(count + 1)} />;
}

// Child just calls the function it was given — doesn't know or care what it does
function Button({ onClick }: { onClick: () => void }) {
  return <button onClick={onClick}>Click me</button>;
}
```

Children as a Prop

- Anything between the opening and closing tag becomes the children prop
  - Type it as React.ReactNode
    - Accepts text, elements, arrays, fragments, null
  - Enables composition
    - Outer component controls layout, inner content is flexible
  - The most common pattern for layout, modal, and panel components
  - Different from passing JSX as a regular prop
    - Children is positional and natural

```tsx
type CardProps = {
  title: string;
  children: React.ReactNode;
};

function Card({ title, children }: CardProps) {
  return (
    <div className="card">
      <h3>{title}</h3>
      <div className="card-body">{children}</div>
    </div>
  );
}

// Used like:
<Card title="Account">
  <p>Balance: $1,234.56</p>
  <Button label="Transfer" onClick={handleTransfer} />
</Card>
```

Hooks

Class Lifecycle Methods

- These are legacy
  - componentDidMount
    - "I just appeared on screen"
    - Set up subscriptions, fetch initial data
  - componentDidUpdate
    - "Something about me changed"
    - re-fetch, sync to new props
  - componentWillUnmount
    - "I'm about to be removed"
    - Clean up subscriptions, cancel timers
  - All three are now replaced by one hook useEffect

```
class UserProfile extends React.Component {
  componentDidMount() {
    // Runs once, after the component is first added to the DOM
    fetch(`/api/users/${this.props.userId}`).then(...);
  }

  componentDidUpdate(prevProps) {
    // Runs after every re-render (except the first)
    if (prevProps.userId !== this.props.userId) {
      fetch(`/api/users/${this.props.userId}`).then(...);
    }
  }

  componentWillUnmount() {
    // Runs once, just before the component is removed from the DOM
    this.subscription?.cancel();
  }
}
```

Hooks

- The pain point this caused
  - Related logic was scattered across three methods
    - Subscribing to a data source in componentDidMount
    - Updating the subscription when props changed in componentDidUpdate
    - And tearing it down in componentWillUnmount
  - Meant one feature lived in three places
    - Add a second subscription with different dependencies and you're now juggling six bits of code across three methods

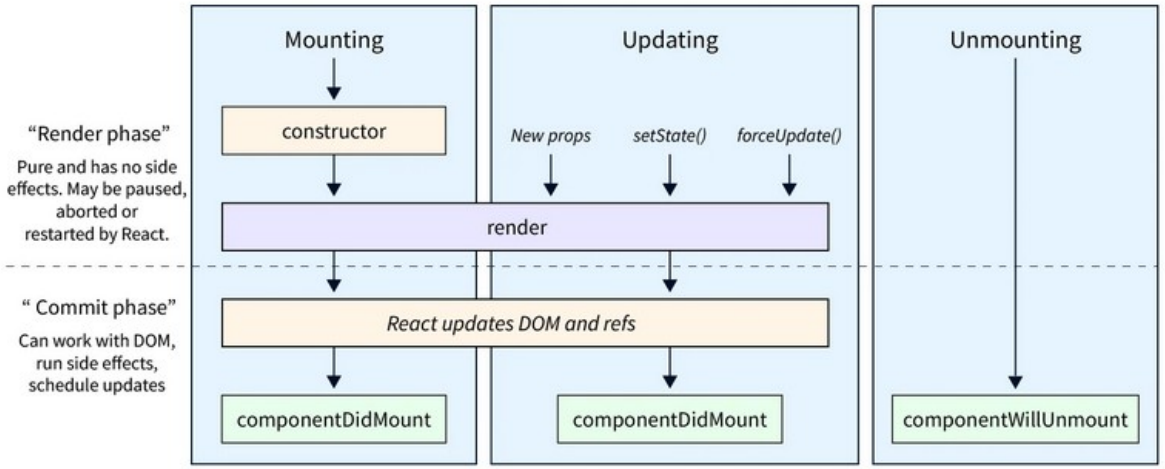


Image Credit: https://www.scaler.com/topics/react/react-hooks/

Hooks

- The problem hooks solve
  - Functional components had no way to remember state between renders
  - No way to run code after the DOM updated
  - Stateful logic was trapped in class components, hard to reuse
  - Class lifecycle methods scatters related logic across
    - componentDidMount
    - componentDidUpdate
    - componentWillUnmount

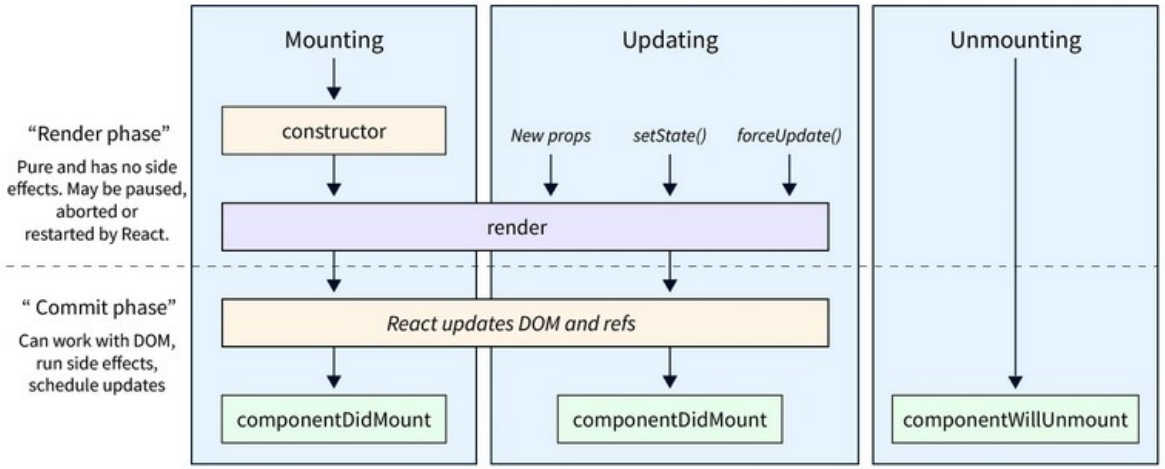


Image Credit: https://www.scaler.com/topics/react/react-hooks/

Hooks

- A hook is
  - A function whose name starts with use
  - Called from inside a component function or another hook
  - Hooks read and write component-local state managed by React
  - React relies on call order to track which state belongs to which hook
  - This is why the rules of hooks exist

```
function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}
```

Hooks

- A hook is
  - A regular JavaScript function that follows two conventions:
  - Its name starts with use
    - useState, useEffect, useMyCustomLogic
  - It's called from inside a React component or another hook.
  - React relies on call order
    - When a component calls useState, then useEffect, then useState again
    - React internally tracks "this component has three hooks: state, effect, state, in that order"
    - On the next render, it expects exactly the same calls in exactly the same order, and it uses the order to match each call to the correct piece of stored state
  -

```
function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}
```

useState

- useState
  - Returns a tuple
    - Current value, setter function
  - Setter triggers a re-render with the new value
  - State is preserved across renders
    - Survives until the component unmounts
  - Each useState call is independent
    - Call it multiple times for multiple values
    - The initial value is used only on the first render
  - Setter function
    - Should be the only way state is updated

```
function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}
```

useState

- useState
  - First render
    - useState(0) runs
    - React allocates a slot for this component, stores 0 in it, returns [0, setCount]
    - setCount is the user defined setter function
  - User clicks
    - setCount(count + 1) runs, which is setCount(1)
    - React updates the stored value to 1 and schedules a re-render
  - Re-render
    - useState(0) runs again
    - But this time, React sees the slot already exists with value 1, ignores the 0 argument, and returns [1, setCount].

```
function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}
```

useState

- useState
  - Treat state as immutable
    - Never mutate; always replace
  - For objects
    - Use spread to create a new object with the change
  - For arrays
    - Use .map, .filter, [...arr, newItem]
    - Never .push, .splice
  - Use the functional setter (setX(prev => ...)) when the new value depends on the old
  - Functional setters protect against stale closures in async code

```
// Wrong — mutating state directly
const [user, setUser] = useState({ name: 'Ada', age: 30 });
user.age = 31;  // React doesn't notice; no re-render

//  Right — replace the object
setUser({ ...user, age: 31 });

//  Functional update — when the new value depends on the old
setCount(prev => prev + 1);
```

useState

- Rule
  - Never modify state in place
    - React decides whether to re-render by comparing old state to new state with Object.is (essentially reference equality)
    - If you mutate an object and pass the same reference back, React sees no change and skips the render
  - The fix is always "create a new value":
    - Object update: { ...obj, field: newValue }
    - Array append: [...arr, newItem]
    - Array remove: arr.filter(x => x.id !== id)
    - Array update: arr.map(x => x.id === id ? { ...x, ...changes } : x)

```
// Wrong — mutating state directly
const [user, setUser] = useState({ name: 'Ada', age: 30 });
user.age = 31;  // React doesn't notice; no re-render

//  Right — replace the object
setUser({ ...user, age: 31 });

//  Functional update — when the new value depends on the old
setCount(prev => prev + 1);
```

useEffect

- UseEffect
  - Synchronizing with the outside world
    - Runs after the render is committed to the DOM
  - For side effects
    - HTTP calls, subscriptions, timers, manual DOM manipulation
  - The dependency array controls when the effect re-runs
    - Returns an optional cleanup function for unsubscribing or cancelling
  - Replaces
    - ComponentDidMount
    - ComponentDidUpdate,
    - componentWillUnmount

```
function UserProfile({ userId }: { userId: string }) {
  const [user, setUser] = useState<User | null>(null);

  useEffect(() => {
    fetch(`/api/users/${userId}`)
      .then(res => res.json())
      .then(setUser);
  }, [userId]);

  return user ? <div>{user.name}</div> : <div>Loading...</div>;
}
```

useEffect

- UseEffect
  - Is for synchronizing your component with something outside React
    - HTTP requests, browser APIs, third-party libraries, web sockets, timers, manual DOM manipulation
  - If something needs to happen as a result of a render but doesn't belong in the render
  - Then it goes in an effect

```
function UserProfile({ userId }: { userId: string }) {
  const [user, setUser] = useState<User | null>(null);

  useEffect(() => {
    fetch(`/api/users/${userId}`)
      .then(res => res.json())
      .then(setUser);
  }, [userId]);

  return user ? <div>{user.name}</div> : <div>Loading...</div>;
}
```

useEffect

- UseEffect
- In the example
  - Component renders with user = null
    - JSX returns Loading... .
  - After commit, the effect runs
    - It fetches data, sets state when the data arrives.
  - State change triggers a re-render
    - JSX returns {user.name} .
  - Effect's dependency array is [userId]
    - The effect re-runs only if userId changes

```
function UserProfile({ userId }: { userId: string }) {
  const [user, setUser] = useState<User | null>(null);

  useEffect(() => {
    fetch(`/api/users/${userId}`)
      .then(res => res.json())
      .then(setUser);
  }, [userId]);

  return user ? <div>{user.name}</div> : <div>Loading...</div>;
}
```

useEffect

- The dependency array
  - The array lists every value from the component scope that the effect uses
  - React re-runs the effect when any listed value changes
    - In the example, when userId changes
  - Empty array []
    - Means run once on mount, cleanup on unmount
  - Omitting the array
    - Means run after every render (rarely what you want)

```
useEffect(() => { ... }, [userId]);     // Runs on mount + when userId changes
useEffect(() => { ... }, []);           // Runs only on mount (and cleanup on unmount)
useEffect(() => { ... });               // Runs after every render — usually a bug
```

useEffect

- Effect cleanup
  - Return a function from the effect
    - React calls it before re-running, and on unmount
  - Clean up
    - timers, subscriptions, event listeners, in-flight requests
  - Without cleanup
    - Memory leaks, "can't update unmounted component" warnings
    - Effect ran once. cleanup runs once
    - Effect ran 5 times, cleanup runs 5 times.
  - Pair every subscription with its teardown

```
useEffect(() => {
  const id = setInterval(() => console.log('tick'), 1000);

  return () => clearInterval(id);  // Cleanup
}, []);

useEffect(() => {
  const controller = new AbortController();

  fetch('/api/data', { signal: controller.signal })
    .then(res => res.json())
    .then(setData);

  return () => controller.abort();
}, []);
```

useEffect

- Effect cleanup
  - Cleanup is the single most-forgotten part of effects
  - the bugs it causes are the kind that compile fine, look fine in development, and crash in production

```
[mount]
  effect runs
  ...
  [some dependency changes]
  cleanup from previous effect runs
  effect runs again
  ...
  [unmount]
  cleanup runs one final time
```

useEffect

- Common effect pitfalls
- Stale closure
  - The effect uses a value from props or state
  - But the dependency array doesn't include it
  - The effect's closure captures the value at the time it was created
    - But never sees later updates.
  - The fix is either to add the value to the dependency array (the right answer)
    - Or to use the functional setter (setCount(prev => prev + 1)) so the latest value is always passed in

```
// Infinite loop — count is in deps, effect updates count
useEffect(() => {
  setCount(count + 1);
}, [count]);  // ❌ Re-runs forever

// Stale closure — count is used but not in deps
useEffect(() => {
  const id = setInterval(() => setCount(count + 1), 1000);
  return () => clearInterval(id);
}, []);  // ❌ count is always 0 inside the closure
```

useEffect

- Common effect pitfalls
- Infinite loop
  - When the effect modifies a value that's in its own dependency array
    - Effect runs → state changes
    - state changes → dependency changed
    - dependency changed → effect runs again
    - effect runs again → state changes again ...
  - The fix is usually to remove the value from the dependency array
    - Or use a functional setter so you don't need to read the value
    - Or restructure the logic so the side effect isn't tied to that piece of state

```
// Infinite loop — count is in deps, effect updates count
useEffect(() => {
  setCount(count + 1);
}, [count]);  // ❌ Re-runs forever

// Stale closure — count is used but not in deps
useEffect(() => {
  const id = setInterval(() => setCount(count + 1), 1000);
  return () => clearInterval(id);
}, []);  // ❌ count is always 0 inside the closure
```

useContext

- useContext
  - Context lets a component reach up the tree for shared data
  - Avoids passing props through layers of intermediate components
  - A Provider sets the value
    - Any descendant can read it with useContext
  - Re-renders all consumers when the provider's value changes
  - Use sparingly
    - Context is for truly global data

```
type Theme = 'light' | 'dark';
const ThemeContext = createContext<Theme>('light');

function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Toolbar />
    </ThemeContext.Provider>
  );
}

function Toolbar() {
  return <Button />;  // Doesn't have to pass theme through
}

function Button() {
  const theme = useContext(ThemeContext);  // Reaches up to the provider
  return <button className={`btn-${theme}`}>Click</button>;
}
```

useContext

- useContext
- Common uses:
  - Theme (light/dark mode)
  - Authenticated user
  - Localization, like current language, translation function
  - Feature flags

```tsx
type Theme = 'light' | 'dark';
const ThemeContext = createContext<Theme>('light');

function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Toolbar />
    </ThemeContext.Provider>
  );
}

function Toolbar() {
  return <Button />;  // Doesn't have to pass theme through
}

function Button() {
  const theme = useContext(ThemeContext);  // Reaches up to the provider
  return <button className={`btn-${theme}`}>Click</button>;
}
```

useContext

- useContext
  - Context isn't a global variable
  - The value is scoped to the provider's subtree
  - Different parts of the tree can have different providers with different values for the same context

```tsx
type Theme = 'light' | 'dark';
const ThemeContext = createContext<Theme>('light');

function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Toolbar />
    </ThemeContext.Provider>
  );
}

function Toolbar() {
  return <Button />;  // Doesn't have to pass theme through
}

function Button() {
  const theme = useContext(ThemeContext);  // Reaches up to the provider
  return <button className={`btn-${theme}`}>Click</button>;
}
```

useReducer

- useReducer
  - When useState gets complex
  - For state that has multiple fields that change together
    - A reducer is a pure function
    - (state, action) => newState
  - Components dispatch actions
    - The reducer decides how state changes
  - Use when
    - Several state fields change together in coordinated ways
    - State transitions follow a clear pattern (loading → success / failure).
    - You want to centralize the update logic instead of scattering setter calls

```
type State = { items: Item[]; loading: boolean; error: string | null };
type Action =
    | { type: 'load_start' }
    | { type: 'load_success'; items: Item[] }
    | { type: 'load_failure'; error: string };

function reducer(state: State, action: Action): State {
    switch (action.type) {
        case 'load_start':   return { ...state, loading: true, error: null };
        case 'load_success': return { items: action.items, loading: false, error: null };
        case 'load_failure': return { ...state, loading: false, error: action.error };
    }
}

const [state, dispatch] = useReducer(reducer, { items: [], loading: false, error: null });
```

useRef

- Provides a mutable box
  - Returns an object with a .current property
    - Mutating .current does not trigger a re-render
  - Two main uses
    - Mutable values that survive renders
    - DOM references
  - The escape hatch from React's "everything is state" model
  - Use only when you genuinely need it

```
// 1. Mutable values that don't trigger re-renders
const renderCount = useRef(0);
useEffect(() => { renderCount.current += 1; });

// 2. Direct DOM access
function FocusInput() {
  const inputRef = useRef<HTMLInputElement>(null);
  useEffect(() => { inputRef.current?.focus(); }, []);
  return <input ref={inputRef} />;
}
```

Hook Rules

- Rule 1
  - Only call hooks at the top level of a component or another hook
    - Not inside conditions
    - Not inside loops
  - Not inside nested functions
- Rule 2
  - Only call hooks from React functions Components, or other hooks
  - Not regular functions, not class components

```
function Component({ condition }) {
    const [a, setA] = useState(1);
    if (condition) {
        const [b, setB] = useState(2);  // Sometimes this hook is called, sometimes not
    }
    const [c, setC] = useState(3);
}
```

Hook Rules

- When a component calls hooks
  - React stores the resulting state in a list
    - In the order the calls happen
  - On the next render
    - React expects the same hooks in the same order
    - It uses position in that list to match each hook call to its stored state
  - This is violated in the example
    - This will produce runtime errors

```
function Component({ condition }) {
    const [a, setA] = useState(1);
    if (condition) {
        const [b, setB] = useState(2);  // Sometimes this hook is called, sometimes not
    }
    const [c, setC] = useState(3);
}
```

Custom Hooks

- Reusable stateful logic
  - A regular function that calls other hooks
  - Name must start with use
  - Lets you extract and reuse stateful logic across components
  - Each call gets its own independent state
    - They don't share
  - The single biggest reason hooks were introduced

```tsx
function useDebouncedValue<T>(value: T, delayMs: number): T {
  const [debounced, setDebounced] = useState(value);

  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delayMs);
    return () => clearTimeout(id);
  }, [value, delayMs]);

  return debounced;
}

// Used like any other hook
function SearchBox() {
  const [query, setQuery] = useState('');
  const debouncedQuery = useDebouncedValue(query, 300);
  // ...
}
```

Custom Hooks

- useDebouncedValue
  - Composes useState and useEffect into a reusable data-fetching primitive
  - Loading, error, and data states managed in one place
  - Cleanup with AbortController to cancel in-flight requests
  - Generic over the response type for type safety

```
function useDebouncedValue<T>(value: T, delayMs: number): T {
  const [debounced, setDebounced] = useState(value);

  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delayMs);
    return () => clearTimeout(id);
  }, [value, delayMs]);

  return debounced;
}

// Used like any other hook
function SearchBox() {
  const [query, setQuery] = useState('');
  const debouncedQuery = useDebouncedValue(query, 300);
  // ...
}
```

Custom Hooks

- useDebouncedValue
  - Just a function
    - Accepts a value and a delay
    - Calls useState and useEffect internally, and returns the debounced value.
    - The component that uses it doesn't see any of the internal state or effect
    - just gets back a value that lags behind the input by delayMs.
  - Each time a component calls useDebouncedValue
    - it gets its own independent state
    - Two components on the page using the same hook with different values are isolated from each other.
    - The hook isn't shared state; it's shared logic

```
function useDebouncedValue<T>(value: T, delayMs: number): T {
  const [debounced, setDebounced] = useState(value);

  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delayMs);
    return () => clearTimeout(id);
  }, [value, delayMs]);

  return debounced;
}

// Used like any other hook
function SearchBox() {
  const [query, setQuery] = useState('');
  const debouncedQuery = useDebouncedValue(query, 300);
  // ...
}
```

Custom Hooks

- useApi
  - Three pieces of state, all related, all managed together
    - The effect handles the entire lifecycle
    - Start loading, fetch, set data or error, clear loading, clean up on unmount or URL change
  - The cleanup function
    - Aborts in-flight fetches if the component unmounts or the URL changes mid-request.
    - Without this, you'd see "set state on unmounted component" warnings.
  - AbortError check prevents treating a deliberate abort as a real error
  - The hook is generic over T
    - Callers tell TypeScript what shape they expect back, and the hook gives them typed data

```
function useApi<T>(url: string): { data: T | null; loading: boolean; error: Error | null } {
    const [data, setData] = useState<T | null>(null);
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState<Error | null>(null);

    useEffect(() => {
        const controller = new AbortController();
        setLoading(true);

        fetch(url, { signal: controller.signal })
            .then(res => res.json())
            .then(setData)
            .catch(err => { if (err.name !== 'AbortError') setError(err); })
            .finally(() => setLoading(false));

        return () => controller.abort();
    }, [url]);

    return { data, loading, error };
}
```

State Management

State Location

- Where to store state
  - Local component state
    - Used by one component only
  - Lifted state
    - Shared by a few related components
  - Context
    - Used by many components across distant parts of the tree
  - External library
    - Large, complex, or app-wide state with sophisticated update patterns
  - Default to the simplest level that works
    - Promote when the simple version starts to hurt

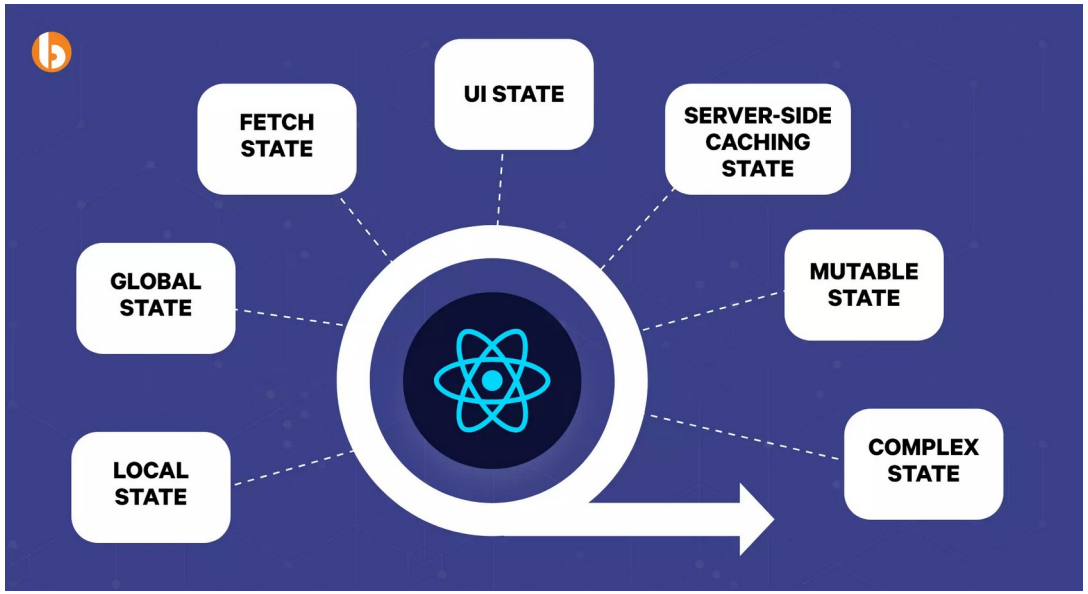


Image Credit:https://skillzam.com/notes/reactjsNotes.html

State Location

- State costs
  - Local state is private, isolated, easy to reason about
    - No coordination needed.
  - Lifted state introduces props that cross components
    - Slightly harder to follow, but still local
  - Context introduces "spooky action at a distance"
    - A value can change anywhere and propagate everywhere
    - Hard to track without tooling.
  - State libraries add concepts (stores, slices, actions, selectors) and a learning curve

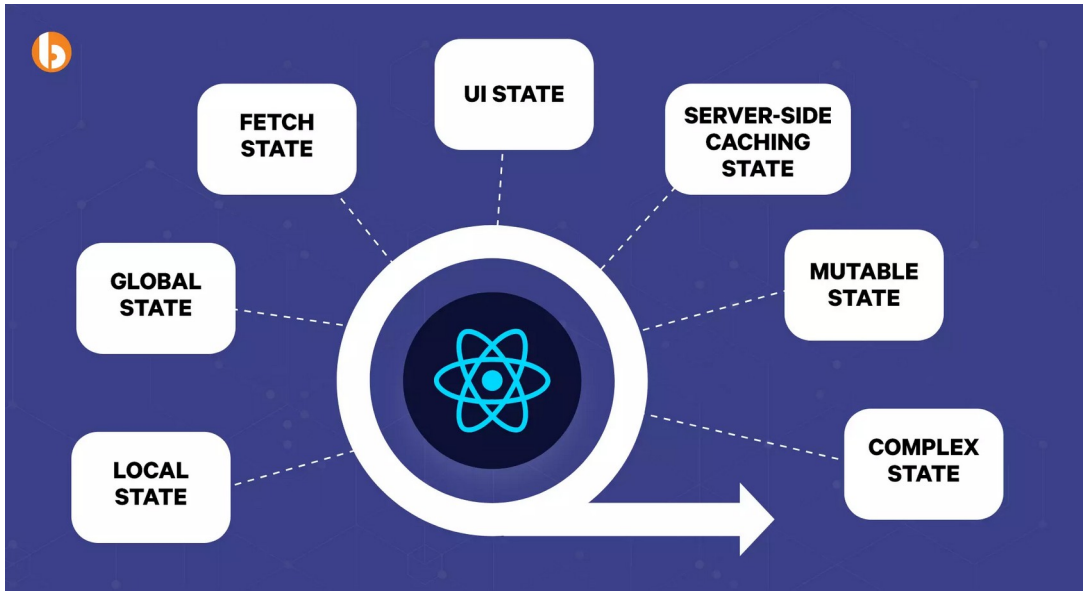


Image Credit:https://skillzam.com/notes/reactjsNotes.html

Local State

- The default
  - State that only one component cares about stays inside that component
  - No coordination, no propagation, no API exposed to other components
  - Form fields, toggle states, hover states, expanded/collapsed flags
  - The vast majority of state in a React app should be local
  - If no other component reads or writes it, it's local

```
function CommentForm() {
  const [text, setText] = useState('');
  const [submitting, setSubmitting] = useState(false);

  return (
    <form onSubmit={handleSubmit}>
      <textarea value={text} onChange={e => setText(e.target.value)} />
      <button disabled={submitting}>Post</button>
    </form>
  );
}
```

Lifting State

- When siblings need to share
  - Two siblings share state by lifting it to their common parent
  - The parent owns it
    - Children receive it via props and report changes via callbacks
  - Single source of truth
    - No possibility of drift
  - The natural consequence of unidirectional data flow
  - This is React's canonical pattern for shared state

```
// Wrong  Each child has its own copy — they get out of sync
<TemperatureInput unit="celsius" />
<TemperatureInput unit="fahrenheit" />

    // RIght Parent owns the state; children render and report changes
    function Calculator() {
        const [temp, setTemp] = useState(0);
        const [unit, setUnit] = useState<'C' | 'F'>('C');

        return (
            <>
                <TemperatureInput unit="C" value={toCelsius(temp, unit)}    onChange={v => { setTemp(v); setUnit('C'); }} />
        <TemperatureInput unit="F" value={toFahrenheit(temp, unit)} onChange={v => { setTemp(v); setUnit('F'); }} />
        </>
    );
    }
```

Lifting State

- The Prop drilling problem
  - Data flows top-down, but most layers don't actually use the prop
  - Every intermediate component declares a prop just to pass it through
  - Refactoring becomes painful
    - Adding a new piece of shared data hits every layer
  - A signal that the data should travel outside the prop chain
  - The fix: context

```
// User data lives at App, but only HeaderUserMenu uses it
<App user={user}>
  <Layout user={user}>
    <Header user={user}>
      <Toolbar user={user}>
        <UserMenuButton user={user}>
          <HeaderUserMenu user={user} />  {/* finally uses it */}
        </UserMenuButton>
      </Toolbar>
    </Header>
  </Layout>
</App>
```

Context

- For data that many components need
  - A provider at the top sets the value
  - Any descendant can read it directly with useContext
  - The intermediate components are oblivious
    - No props threaded through
  - Best for truly global, slowly-changing data
  - Costs
    - Re-renders propagate to all consumers
    - Harder to trace updates

```tsx
type AuthState = { user: User | null; loading: boolean };
const AuthContext = createContext<AuthState | null>(null);

function App() {
  const [state, setState] = useState<AuthState>({ user: null, loading: true });
  return (
    <AuthContext.Provider value={state}>
      <Layout>...</Layout>
    </AuthContext.Provider>
  );
}

// Anywhere in the tree
function HeaderUserMenu() {
  const auth = useContext(AuthContext);
  return <span>{auth?.user?.name}</span>;
}
```

Context

- When context is right
  - The data is needed by many components in different parts of the tree
  - The data changes infrequently
    - Theme, current user, locale
  - The intermediate components have no business knowing about the data

```tsx
type AuthState = { user: User | null; loading: boolean };
const AuthContext = createContext<AuthState | null>(null);

function App() {
  const [state, setState] = useState<AuthState>({ user: null, loading: true });
  return (
    <AuthContext.Provider value={state}>
      <Layout>...</Layout>
    </AuthContext.Provider>
  );
}

// Anywhere in the tree
function HeaderUserMenu() {
  const auth = useContext(AuthContext);
  return <span>{auth?.user?.name}</span>;
}
```

Context

- When context is wrong
  - The data changes very frequently
    - Every change re-renders every consumer, regardless of which part of the value they actually use
  - The data is only used by a small subtree
    - Just lift it to the root of that subtree
  - You're using it as a global variable for convenience
    - Context is scoped to a provider
    - It's not for arbitrary globals

```tsx
type AuthState = { user: User | null; loading: boolean };
const AuthContext = createContext<AuthState | null>(null);

function App() {
  const [state, setState] = useState<AuthState>({ user: null, loading: true });
  return (
    <AuthContext.Provider value={state}>
      <Layout>...</Layout>
    </AuthContext.Provider>
  );
}

// Anywhere in the tree
function HeaderUserMenu() {
  const auth = useContext(AuthContext);
  return <span>{auth?.user?.name}</span>;
}
```

Context + Reducer

- Mini state container
  - The reducer defines the legal state transitions
  - The provider holds the state and exposes state + dispatch
  - Components dispatch actions
    - They don't poke at state directly
  - Clear, testable, predictable

```
type AuthAction =
    | { type: 'login_start' }
    | { type: 'login_success'; user: User }
    | { type: 'login_failure'; error: string }
    | { type: 'logout' };

function authReducer(state: AuthState, action: AuthAction): AuthState {
    switch (action.type) {
        case 'login_start':   return { ...state, loading: true };
        case 'login_success': return { user: action.user, loading: false };
        case 'login_failure': return { user: null, loading: false };
        case 'logout':        return { user: null, loading: false };
    }
}

const AuthContext = createContext<{ state: AuthState; dispatch: Dispatch<AuthAction> } | null>(null);
```

Context + Reducer

- The combination
  - Instead of putting raw state in context
  - We use (state, dispatch) pair
    - State is read with useContext actions
    - The reducer enforces what state transitions are legal.
  - The example
    - The auth state has multiple fields that change together (user, loading, error)
    - A reducer keeps them coordinated
    - The transitions are well-defined
    - There's a clear "login flow" with start, success, failure stages
    - Components elsewhere in the tree can dispatch actions without knowing how the state is structured.
    - They just say dispatch({ type: 'logout' })

```
type AuthAction =
    | { type: 'login_start' }
    | { type: 'login_success'; user: User }
    | { type: 'login_failure'; error: string }
    | { type: 'logout' };

function authReducer(state: AuthState, action: AuthAction): AuthState {
    switch (action.type) {
        case 'login_start':   return { ...state, loading: true };
        case 'login_success': return { user: action.user, loading: false };
        case 'login_failure': return { user: null, loading: false };
        case 'logout':        return { user: null, loading: false };
    }
}

const AuthContext = createContext<{ state: AuthState; dispatch: Dispatch<AuthAction> } | null>(null);
```

Controlled vs. Uncontrolled Components

- Differences
  - Controlled
    - State is the source of truth; React drives the input
    - Gives you validation, formatting, conditional logic in real time
  - Uncontrolled
    - The DOM is the source of truth; React peeks when needed
    - Aimpler for write-once forms and integrating with non-React code
  - Default to controlled
    - Use uncontrolled when you have a reason

```
// Controlled — React owns the value
<input value={text} onChange={e => setText(e.target.value)} />

// Uncontrolled — DOM owns the value, React reads it via ref on demand
const inputRef = useRef<HTMLInputElement>(null);
<input ref={inputRef} defaultValue="initial" />
// later: inputRef.current?.value
```

Controlled vs. Uncontrolled Components

- Controlled
  - React's state and the DOM are kept in sync
    - Every keystroke updates state, the state drives the DOM via the value prop
    - The benefit is that state is always live and accurate
    - You can validate as the user types, format the value, disable the submit button when invalid, or react to changes anywhere else in the app
- Uncontrolled
  - The DOM holds the value
    - React doesn't track it on every keystroke
    - You set an initial value with defaultValue, and you only read the current value when you need it
    - Fine for forms where you don't need real-time validation

```
// Controlled — React owns the value
<input value={text} onChange={e => setText(e.target.value)} />

// Uncontrolled — DOM owns the value, React reads it via ref on demand
const inputRef = useRef<HTMLInputElement>(null);
<input ref={inputRef} defaultValue="initial" />
// later: inputRef.current?.value
```

Server State vs. Client State

- Client state
  - Owned by the browser
    - Toggles, drafts, selections, in-memory data
- Server state
  - Owned by the backend
  - Server state is never truly fresh
    - it can change while you're looking at it
    - Caching, deduplication, refetching, invalidation are server-state concerns

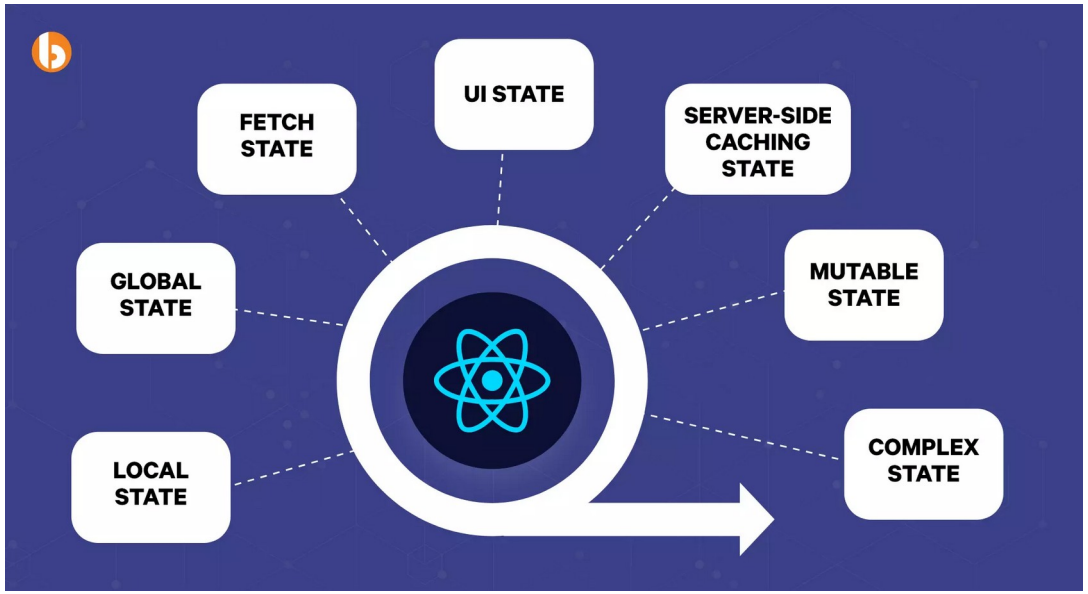
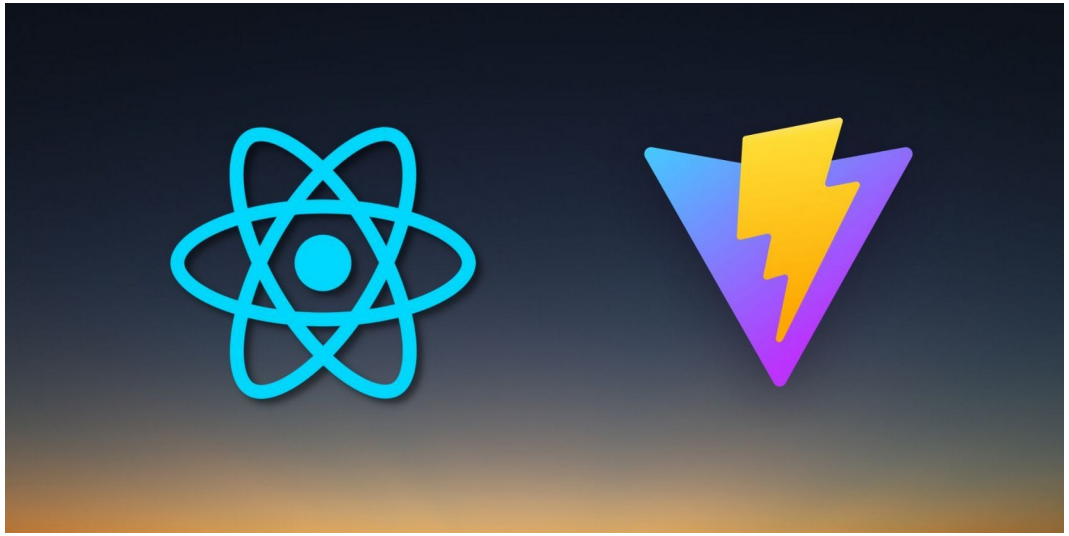


Image Credit:https://skillzam.com/notes/reactjsNotes.html

Project Structure

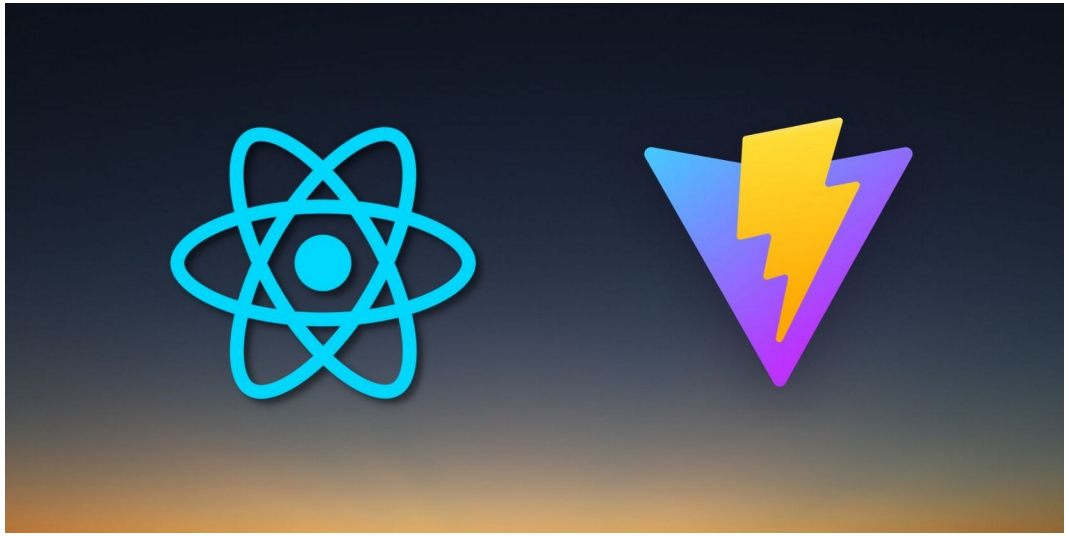
Vite

- Why Vite
  - Fast: dev server starts in milliseconds, not seconds
  - Simple: sensible defaults
    - No config needed for React + TypeScript work
  - Modern
    - Uses native browser ES modules in dev, esbuild + Rollup for production
  - Replaces Create React App
    - CRA is officially deprecated as of 2023
  - Not a framework
    - Pure SPA tooling; no SSR, no routing baked in



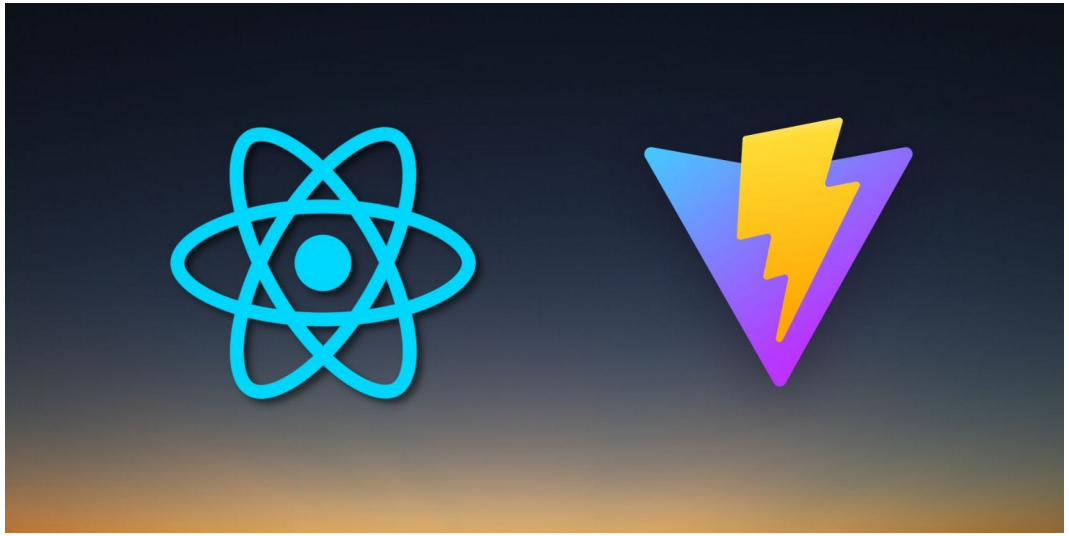
Vite

- Why Vite
  - Current default for pure SPA work
  - Created by Evan You (the creator of Vue)
  - Famework-agnostic
    - Used widely with React, Vue, Svelte, and others.
- The key innovations
  - In development
    - Vite serves files using native ES modules.
    - The browser asks for App.tsx
    - Vite transforms and serves it, the browser asks for its imports, Vite serves those.
    - There's no upfront bundling. The dev server starts essentially instantly.



Vite

- Why Vite
  - For production
    - Vite uses Rollup to produce optimized bundles
    - The output is fully optimized for the browser
    - minified, code-split, tree-shaken.
  - TypeScript and JSX work out of the box.
    - No Babel config to write



Scaffolding a Project

- Creating a new project
  - One command to scaffold a new project
    - The react-ts template gives you React + TypeScript pre-configured
  - npm install pulls dependencies
  - npm run dev starts the dev server
    - typically on http://localhost:5173
  - Open the URL to see the app is running

```
npm create vite@latest my-app -- --template react-ts

cd my-app
npm install
npm run dev
```

Scaffolding a Project

- Creating a new project
  - npm create vite@latest
    - Runs Vite's project creator, which is a small CLI that asks a few questions and writes files.
    - my-app: the directory name and project name
  - The double dash tells npm
    - "everything after this is for the underlying command, not for npm itself"
  - --template react-ts
    - Picks the React + TypeScript template
    - Vite has templates for Vue, Svelte, Lit, and others
    - We want this one.

```
npm create vite@latest my-app -- --template react-ts

cd my-app
npm install
npm run dev
```

Project Layout

- Structure
  - index.html is the real entry point
    - At the project root, not in public/
  - src/main.tsx is the JavaScript entry
    - Mounts React into a DOM node
  - src/App.tsx is your root component
  - vite.config.ts is where you configure plugins and dev server options
  - public/ is for static files
    - Referenced by URL, not imported

```
my-app/
├── public/              ← static assets served as-is (favicon, etc.)
├── src/
│   ├── App.tsx          ← root component
│   ├── main.tsx         ← entry point: mounts <App /> into the DOM
│   ├── App.css
│   ├── index.css
│   └── assets/
├── index.html           ← real HTML entry point — not in /public
├── package.json
├── tsconfig.json
├── vite.config.ts
└── .gitignore
```

Project Layout

- Structure
  - index.html is at the root, not in public/
    - Different from CRA, which hid index.html inside public/ and treated it as a template
    - In Vite, index.html is treated as a real source file
    - The entry point of your application from the browser's perspective
    - Vite processes it like any other source file
    - You can <script type="module" src="/src/main.tsx"> directly in it.
  - src/main.tsx is the JavaScript entry
    - This is the file that calls ReactDOM.createRoot(...).render(<App />)
    - It's small and you'll rarely touch it
    - It exists because React needs an explicit handoff point from "vanilla JavaScript" to "React-controlled DOM."

```
my-app/
├── public/              ← static assets served as-is (favicon, etc.)
├── src/
│   ├── App.tsx          ← root component
│   ├── main.tsx         ← entry point: mounts <App /> into the DOM
│   ├── App.css
│   ├── index.css
│   └── assets/
├── index.html           ← real HTML entry point — not in /public
├── package.json
├── tsconfig.json
├── vite.config.ts
└── .gitignore
```

Project Layout

- Structure
  - src/App.tsx is the root of your component tree
    - Everything else is a descendant
    - Most projects build out the tree from here.
  - public/ is for static files that should be served as-is
    - Anything in public/ is copied verbatim to the build outputthat's src/.
  - vite.config.ts is where you customize Vite
    - The default works for most projects

```
my-app/
├── public/              ← static assets served as-is (favicon, etc.)
├── src/
│   ├── App.tsx          ← root component
│   ├── main.tsx         ← entry point: mounts <App /> into the DOM
│   ├── App.css
│   ├── index.css
│   └── assets/
├── index.html           ← real HTML entry point — not in /public
├── package.json
├── tsconfig.json
├── vite.config.ts
└── .gitignore
```

Project Layout

- package.json and dependencies
  - scripts
    - What npm run <name> actually executes
  - dependencies
    - Needed at runtime like React itself
  - devDependencies
    - Needed only during development
    - Vite, TypeScript, ESLint
  - package-lock.json records exact versions for reproducible installs
    - npm install <package> adds an entry
    - npm install <package> --save-dev puts it under devDependencies

```
{
  "name": "my-app",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build",
    "preview": "vite preview",
    "lint": "eslint ."
  },
  "dependencies": {
    "react": "^19.0.0",
    "react-dom": "^19.0.0"
  },
  "devDependencies": {
    "vite": "^5.0.0",
    "typescript": "^5.5.0",
    "@vitejs/plugin-react": "^4.3.0"
  }
}
```

TypeScript Configuration

- TS config
  - strict: true
    - Enable all strict type-checking
    - Non-negotiable for serious projects
  - jsx: "react-jsx"
    - Modern JSX transform, no need to import React in every file
  - target and lib
    - Which JavaScript features and browser APIs are available
  - The Vite template configures this correctly
    - You'll rarely edit it
    - TypeScript runs only in your editor and during build, not at runtime

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "ESNext",
    "jsx": "react-jsx",
    "strict": true,
    "moduleResolution": "bundler",
    "lib": ["ES2022", "DOM", "DOM.Iterable"]
  },
  "include": ["src"]
}
```

Environment Variables

- Env
  - Vite reads .env files at build time
    - Inlines values into the bundle
  - Variables exposed to the browser must start with VITE_
    - Safety guard against leaking server-only secrets
    - Available as import.meta.env.VITE_*
  - Don't put secrets here, everything in the bundle is public
  - .env.local for developer-specific overrides
    - gitignored by default

```
# .env.development
VITE_API_BASE_URL=http://localhost:8080/api
VITE_AUTH_AUTHORITY=http://localhost:9000
VITE_AUTH_CLIENT_ID=spa-client

# .env.production
VITE_API_BASE_URL=https://api.example.com
VITE_AUTH_AUTHORITY=https://auth.example.com
VITE_AUTH_CLIENT_ID=spa-client-prod
```

```
// Access in code
const apiBaseUrl = import.meta.env.VITE_API_BASE_URL;
```

The Build

- Bundle
  - Output is static files
    - HTML, JS, CSS, assets
    - Filenames include content hashes for aggressive caching
  - Code is minified, tree-shaken, and code-split automatically
  - Deploy by uploading dist/ to any static host
    - CDN, S3, Spring Boot, nginx
  - npm run preview serves the production build locally for testing

```
npm run build
```

```
dist/
├── index.html                ← processed, references hashed bundles
├── assets/
│   ├── index-a3f7b9c.js      ← bundled, minified, content-hashed
│   ├── index-7e2a1b8.css
│   └── ... (chunks)
└── favicon.ico
```

The Build

- Bundle
  - Content-hashed filenames
    - The a3f7b9c in index-a3f7b9c.js is a hash of the file's contents
    - When the file changes, the hash changes, so the filename changes
    - This means you can cache these files forever in the browser becuse they'll never give stale content, because changes produce a new filename.
    - Only index.html (which references the current hashes) needs to be uncacheable.
  - Tree-shaking.
    - The bundler removes code that isn't actually used.
    - If you import one function from a 100-function library, only that function ends up in the bundle.

```
npm run build
```

```
dist/
├── index.html                ← processed, references hashed bundles
├── assets/
│   ├── index-a3f7b9c.js      ← bundled, minified, content-hashed
│   ├── index-7e2a1b8.css
│   └── ... (chunks)
└── favicon.ico
```

The Build

- Bundle
  - Code-splitting
    - For larger apps, Vite automatically splits the bundle into chunks that load lazily as needed
  - npm run preview runs a local static server against the production build
    - Useful for sanity-checking that the production bundle works the same as the dev server.

```
npm run build
```

```
dist/
├── index.html                ← processed, references hashed bundles
├── assets/
│   ├── index-a3f7b9c.js      ← bundled, minified, content-hashed
│   ├── index-7e2a1b8.css
│   └── ... (chunks)
└── favicon.ico
```

Questions?